

Kernel Recipes

Multipath TCP:

When one path is not enough

Matthieu Baerts (NGI0) • Kernel Recipes 26 • 23.09.2026





- What is MPTCP?
- How to use MPTCP on Linux today?
- A long road to get MPTCP upstream
- Current status & next steps

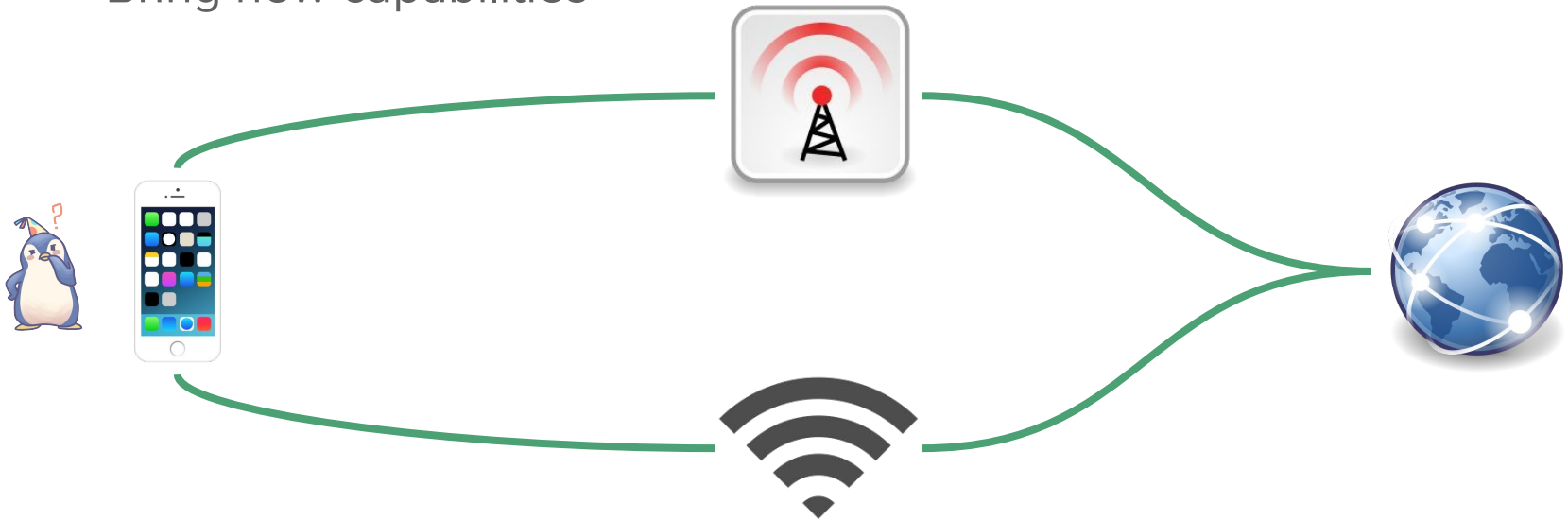
What is MPTCP?



Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Bring new capabilities

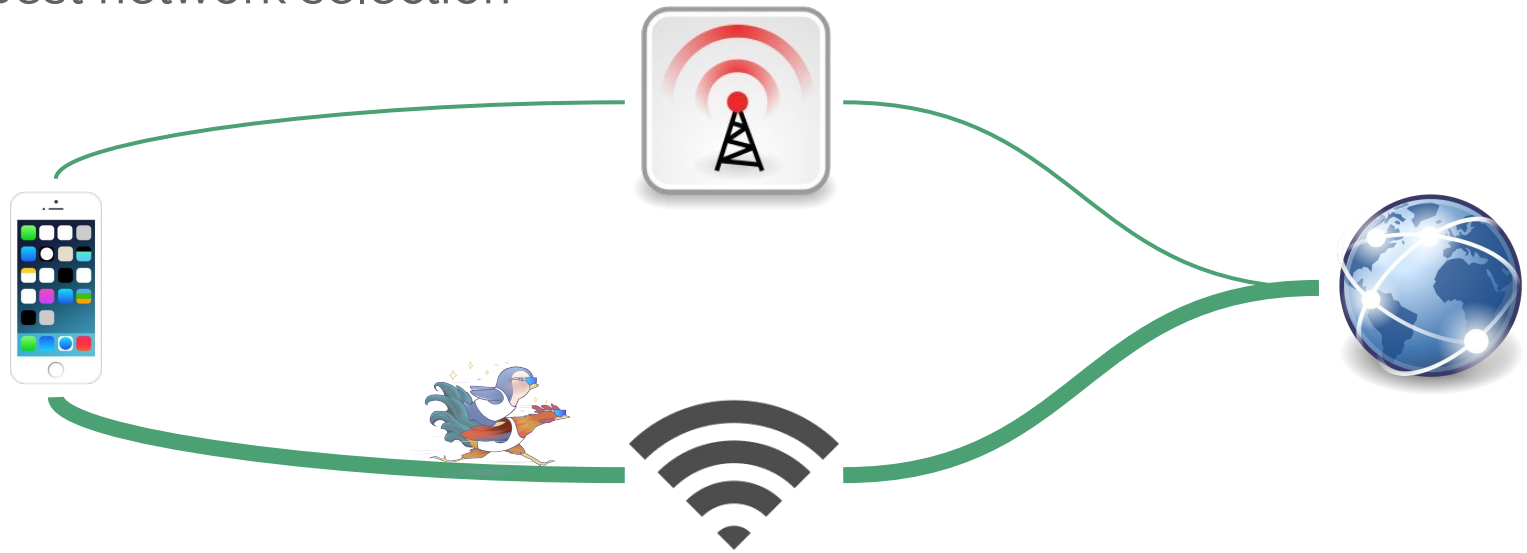


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Best network selection

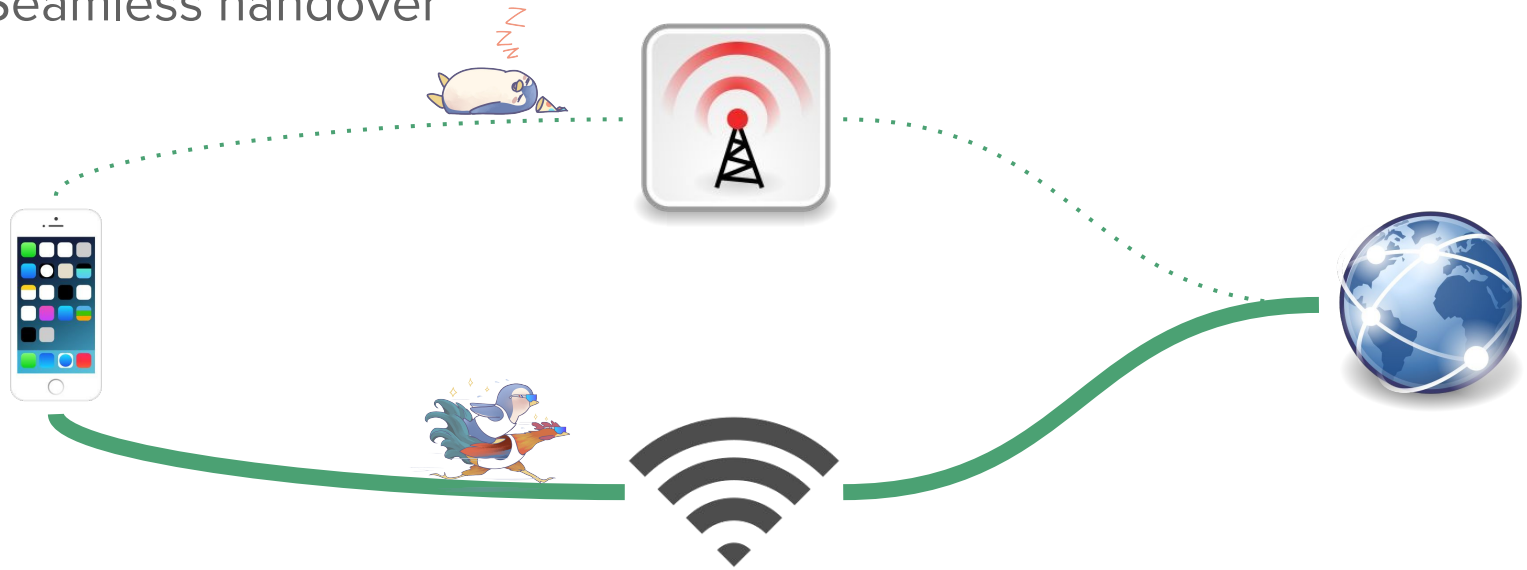


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Seamless handover

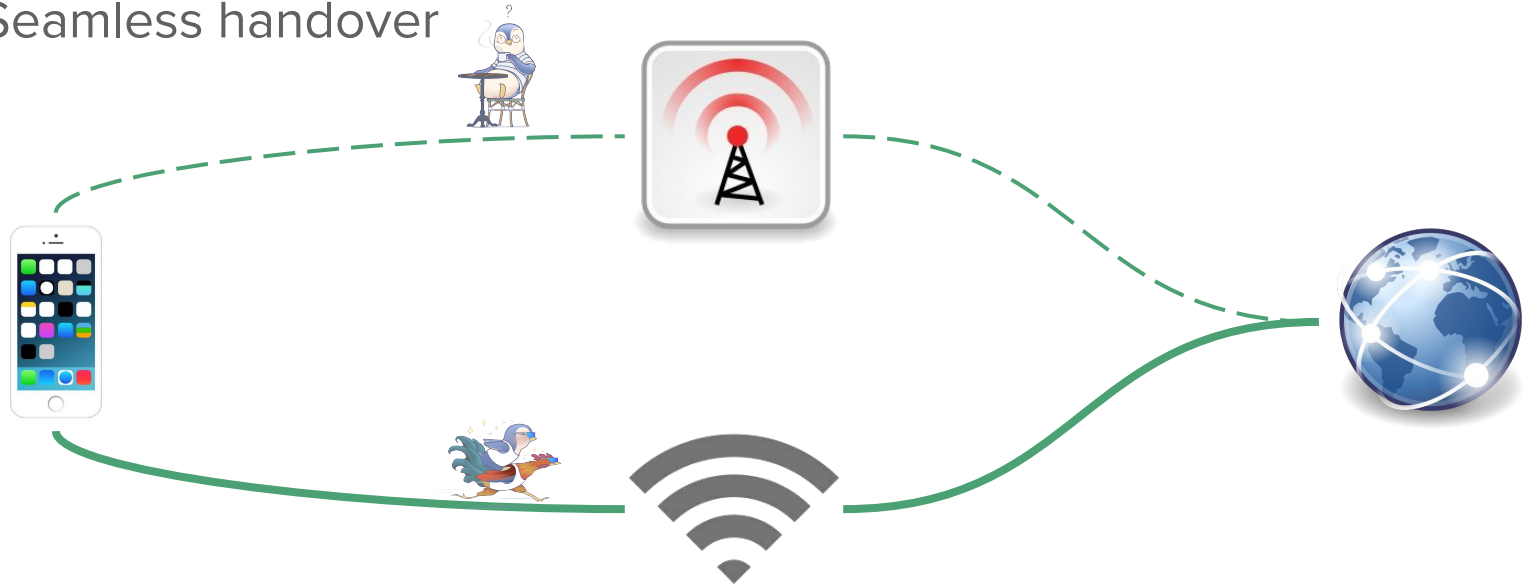


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Seamless handover

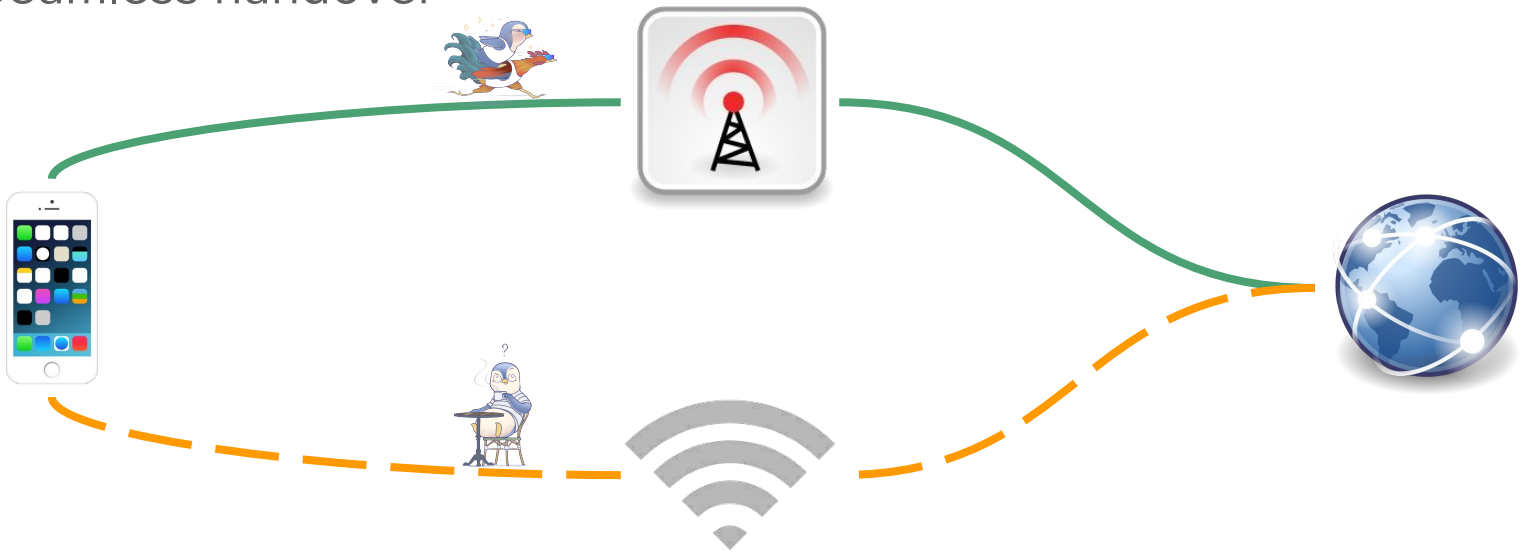


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Seamless handover

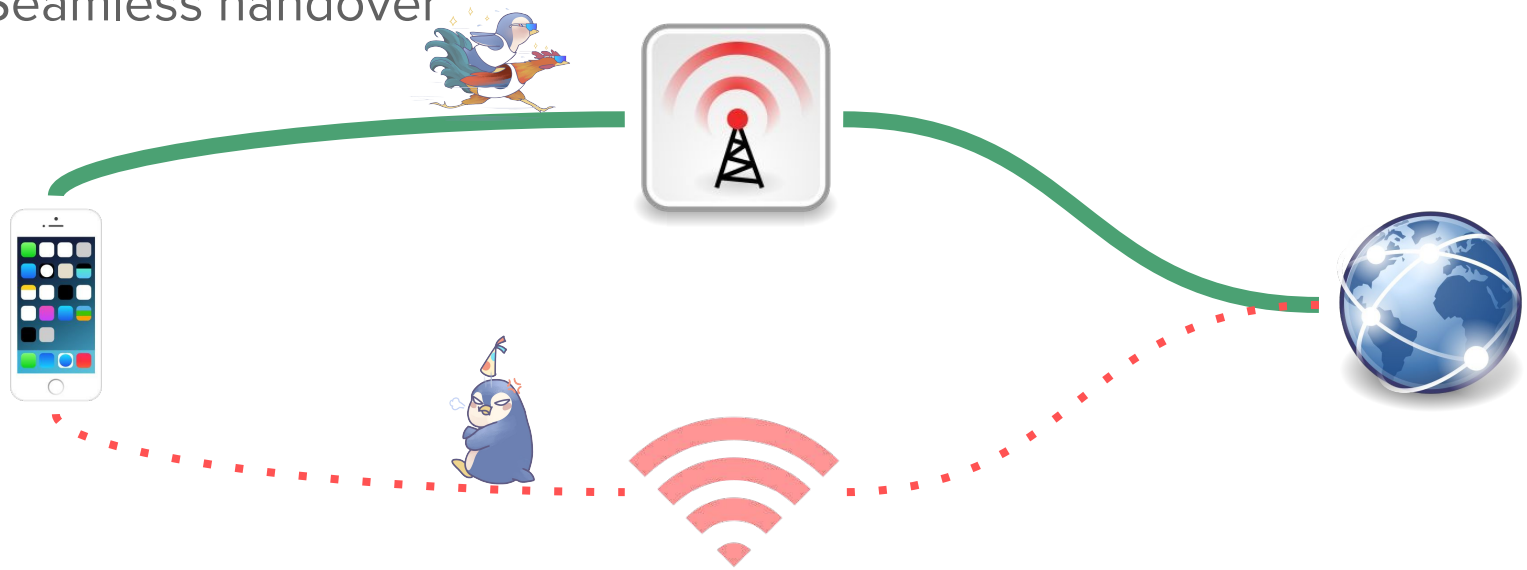


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Seamless handover

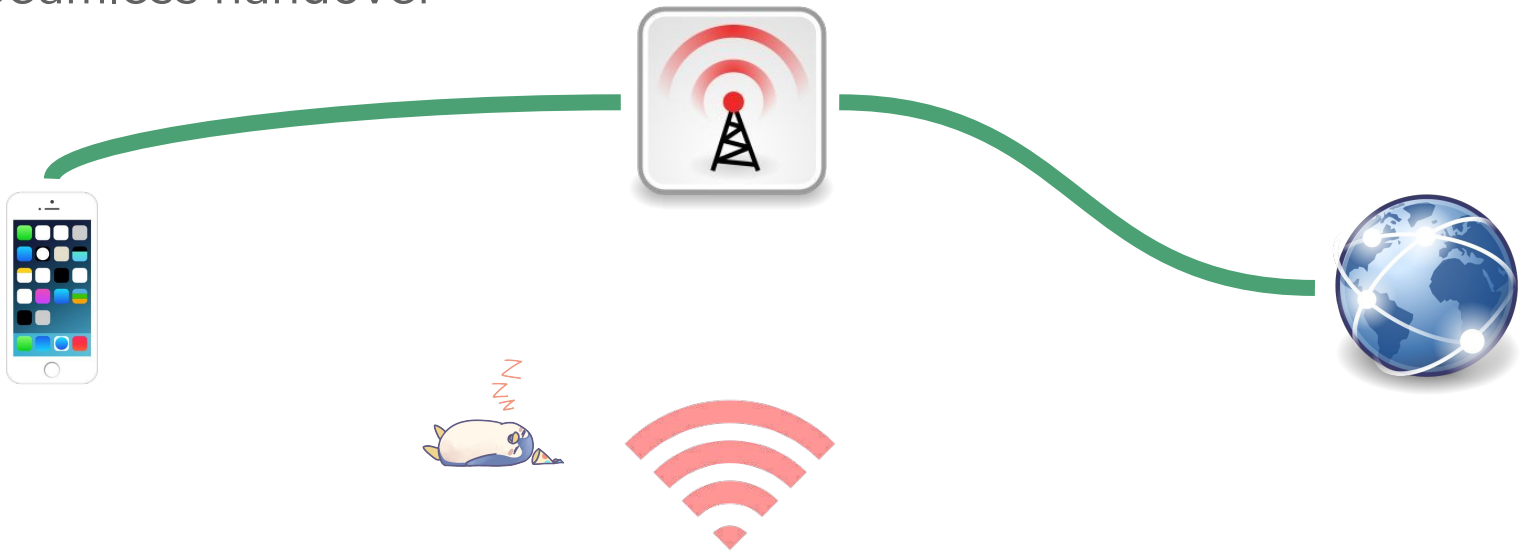


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Seamless handover

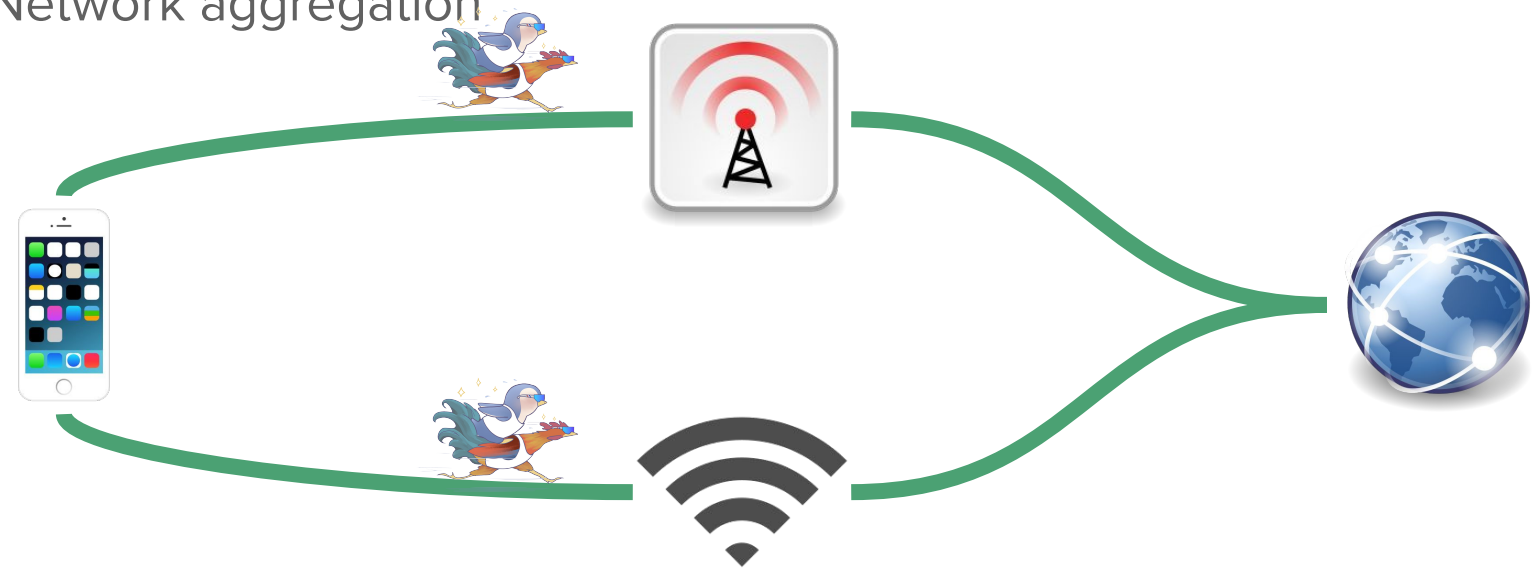


Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Exchange data for a single connection over different paths, simultaneously

⇒ Network aggregation



Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Multipath TCP (MPTCP – RFC-8684)

Use cases:

- Mobile devices:
  - “walk-out” scenario



- Home Gateways:
  - combine networks, e.g. DSL + cellular or low Earth orbit satellites
- Data centres:
  - fast recoveries, select best paths, aggregation



Concept: Subflow and Fallback

- **Subflow**: Each path of an MPTCP connection. A subflow is a regular TCP connection carrying extra options in the TCP header.

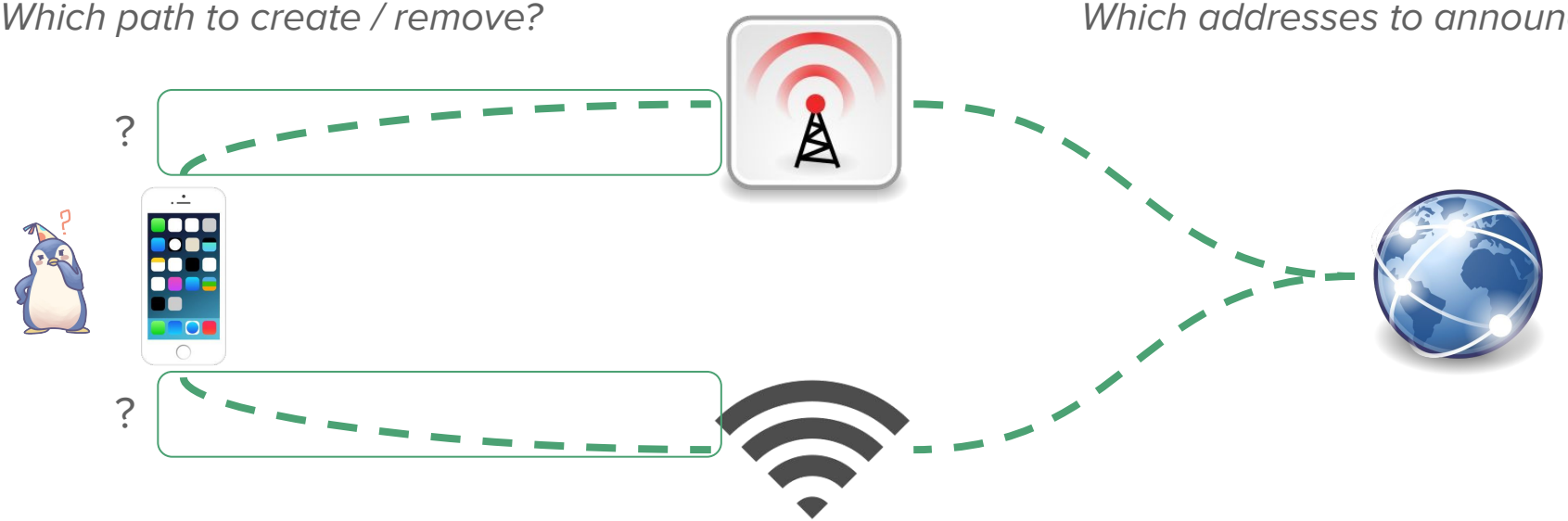
- **Fallback**: If the other host does not support MPTCP, or in case of a middlebox intercepting TCP connections, there will be fallbacks to TCP. The MPTCP protocol is complex to cope with various middleboxes.

Concept: Path Manager

Typically, different needs for the clients and servers:

Which path to create / remove?

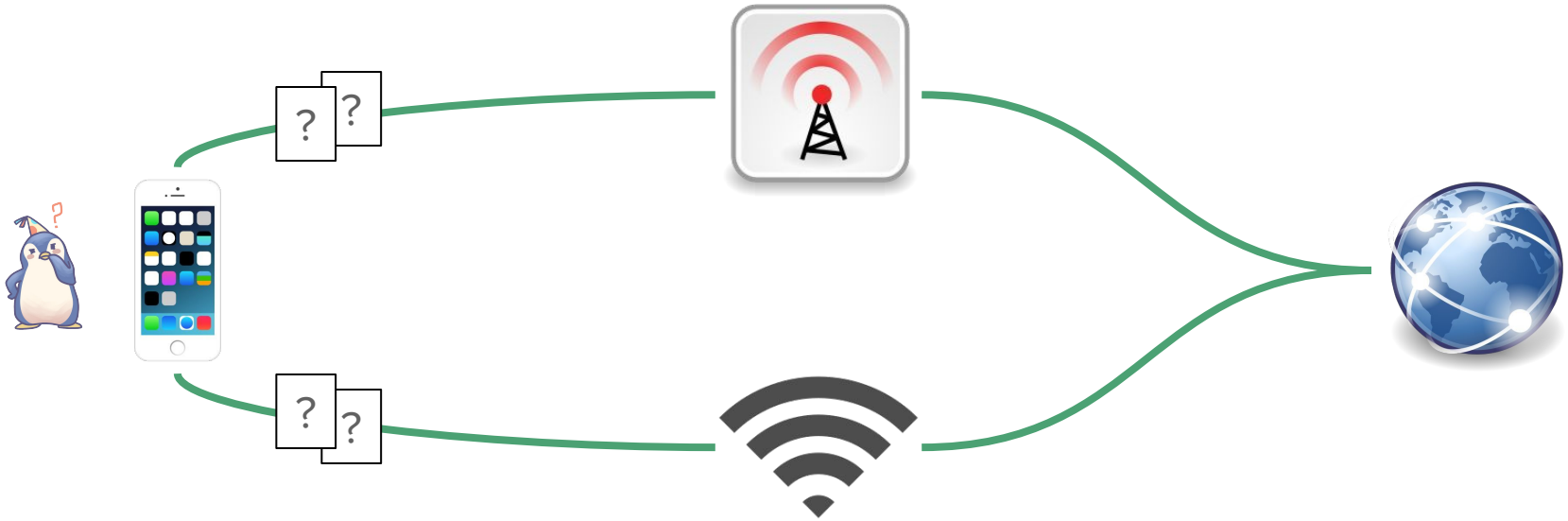
Which addresses to announce?



Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Concept: Packet Scheduler

On which available path packets will be sent? Reinject to another path?



Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

How to use MPTCP on Linux today?



MPTCP on Linux



- The complexity is handled by the kernel
- Opt-in (with possibilities to force apps to use MPTCP):

```
socket(AF_INET(6), SOCK_STREAM, IPPROTO_MPTCP);
```

- Minimal behaviour changes for apps compared to TCP
- Path-Manager configured via userspace, e.g. manually:

```
ip mptcp endpoint add <IP address> dev <interface> <type>
```

MPTCP on Linux



- Some tools can automatically set up the MPTCP endpoints, e.g. **`NetworkManager`** and **`mptcpd`**

- Some apps natively support MPTCP, e.g. cURL, HAProxy, Apache Server, Lighttpd, NGINX, systemd sockets, Go apps (enabled by default on the server side), etc. Check mptcp.dev/apps.html

- Possibilities to force using MPTCP, e.g. `mptcpize` (`LD_PRELOAD`), `mptcpify` (`eBPF/BCC`), `GODEBUG=multipathtcp=1`, SystemTAP, etc. Check mptcp.dev/setup.html

MPTCP on Linux



- Most Linux distributions have MPTCP support enabled, and **`mptcpd`** packages, including specialised ones like OpenWrt, RPiOS, HAOS.

- Debugging tools supports MPTCP, e.g **`ss -M`**, **`ip mptcp`**, **`nstat`**, **`tcpdump`**, **`ptcpdump`**, WireShark

- Server: an MPTCP listen socket will create a TCP socket if the client didn’t request MPTCP: good to have MPTCP enabled by default!

Demo



A long road to get MPTCP upstream



A bit of history



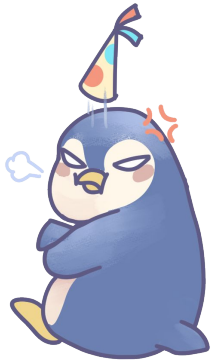
- MPTCP started to appear upstream in v5.6 (Jan. 2020)
- But started as a prototype in 2008/2009 🇧🇪 🍟 🍻
- MPTCPv0 RFC published in January 2013
- Used in production on servers having millions of clients

Maintaining a fork



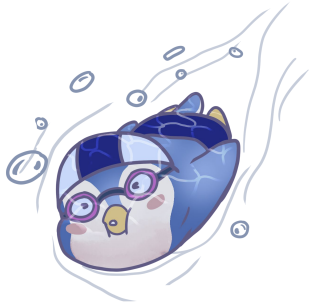
- It is easy to fork ...
  - but you will pay for it!
- The Linux kernel is big, complex, very active
- The fork was quite invasive:
  - 21k lines in total
  - 2.5k in TCP / IP / ... with many “if (mptcp)”
  - With duplicated functions adapted for MPTCP case

The fork couldn’t be accepted upstream



- Linux TCP is highly optimized
- New implementation cannot affect existing TCP stack:
  - No performance regressions
  - Maintainable and possibility to disable it
  - Can be extended via the userspace
- Cannot take the initial fork:
  - Built to support experiments and rapid changes but not generic enough
  - Special purpose implementation of MPTCP

Upstreaming: requirements



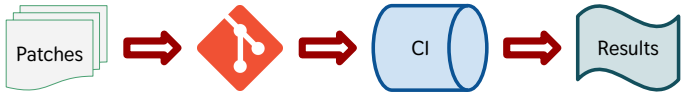
- Rewriting (almost) everything from scratch
- A different concept: introduction of “MPTCP socket”
- Minimal differences in TCP code thanks to TCP ULP (+ SKB ext)
- Carefully review and detail modifications in TCP stack
- APIs to extend the path-manager and the scheduler
- And … A lot of work!

Current status & next steps



Current status: general

- Supports most of the protocol features: multiple subflows, announce addresses and priority, fast close, reset reasons, etc.
- Info from **`MIB`** counters (`nstat`), **`INET_DIAG`** interface (`ss`) and **`MPTCP_INFO`** / **`MPTCP_FULL_INFO`** socket options
- CI validating patches sent on the ML (virtme-ng, GitHub Actions, etc.)



Current & Future: socket options

- Current: Supports most common socket options: **`SO`**, **`IP`**, **`IPV6`**, **`TCP`**:
  - Imitating TCP's behaviour
  - But adapted to MPTCP case:
    - Inherit the behaviour on all subflows, including future ones? e.g. KeepAlive
    - Or only on the first one? e.g. TCP FastOpen
  - Still possible to change the per-subflow behaviour with eBPF
- Future: Support more uncommon ones, e.g. `(MP)TCP_REPAIR`

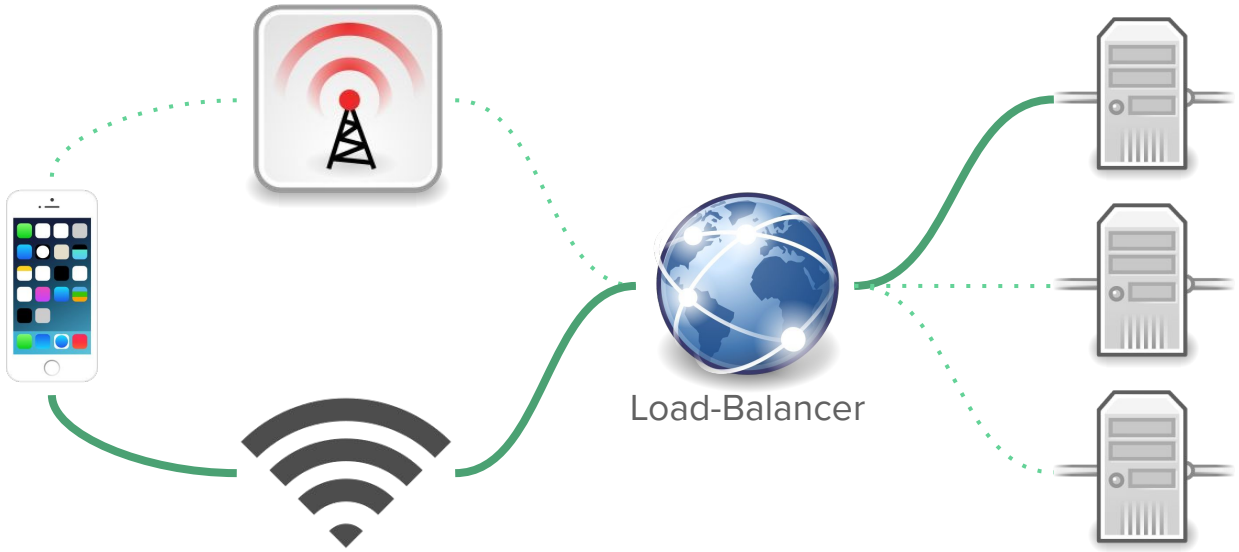
Current status: path managers

- *In-kernel*: Global settings per network namespace: e.g. via **`ip mptcp`**
  - Set endpoints: IP addresses, flags (client-server sides, backup, fullmesh)
  - Set limits: max subflows to establish or accept
  - Monitor connections: created, established, closed, announced, etc.
- *Userspace*: Per connection: e.g. via **`mptcpd`**
  - Reacting to “**`events`**” by sending “**`commands`**”

⇒ Communications: using Netlink

Current status: PM: Deployment behind a Load-Balancer

- Initial path: with a random server behind a stateless load-balancer

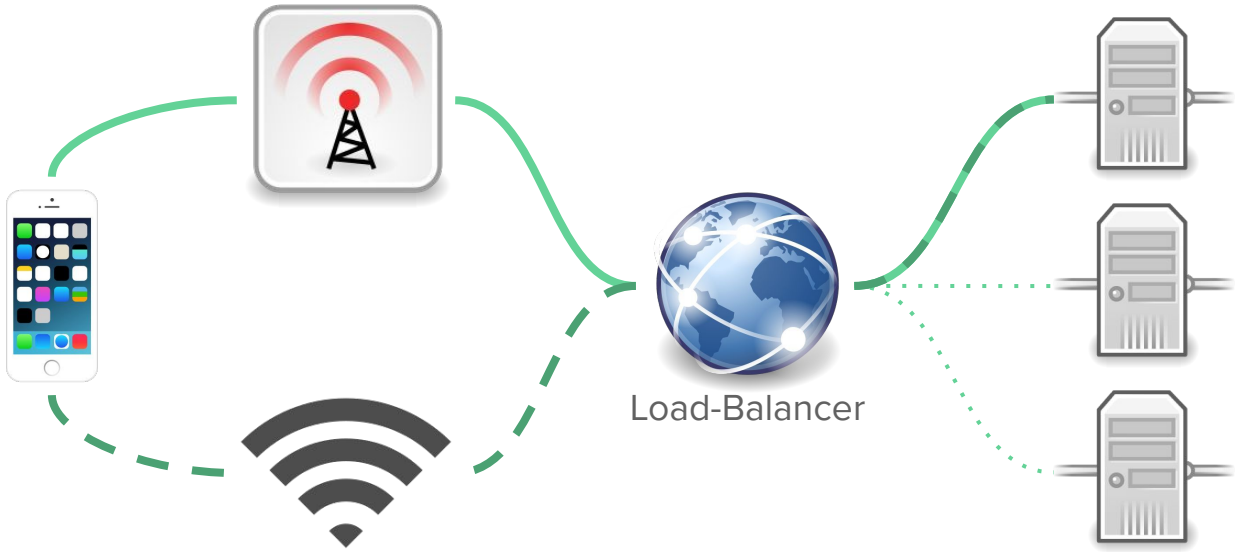


Anycast: each server is reusing the same public IP

Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Current status: PM: Deployment behind a Load-Balancer

- Additional paths: how a stateless load-balancer can pick the same server?



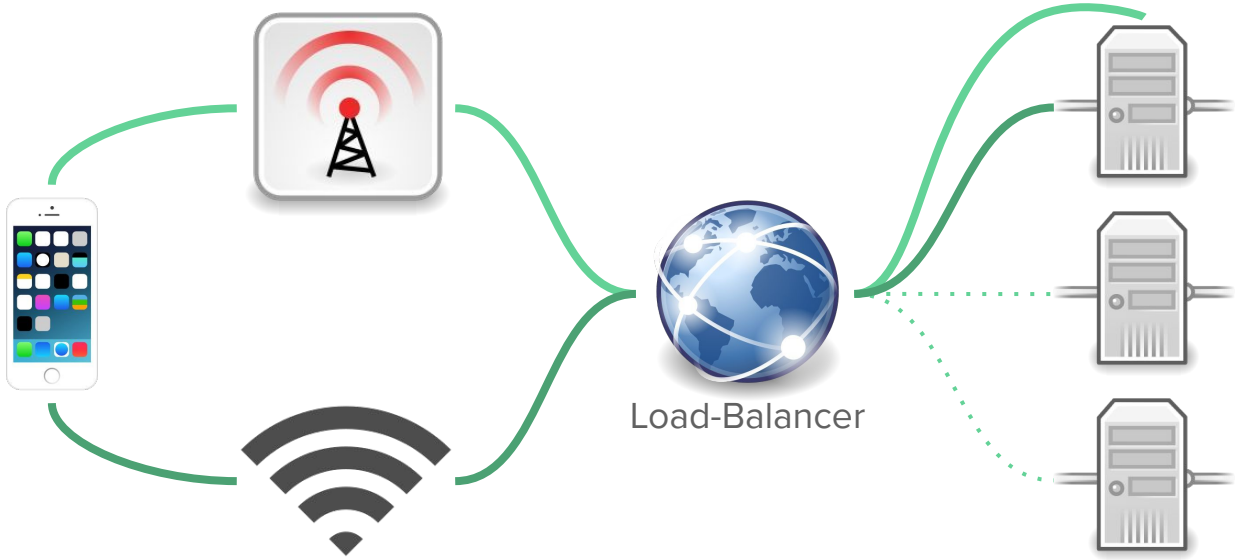
Anycast: each server is reusing the same public IP

Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Current status: PM: Deployment behind a Load-Balancer

- Additional paths: how a stateless load-balancer can pick the same server?

  ⇒ Servers: tell client not to use initial address and announce a new one.



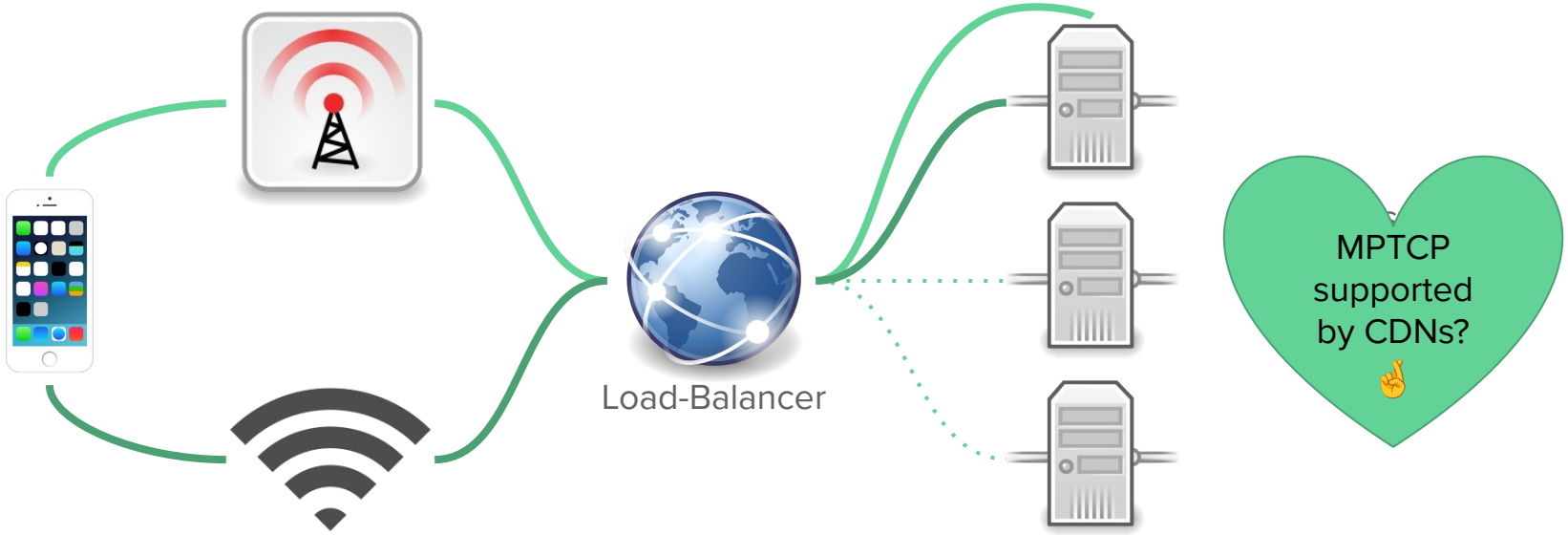
Anycast: each server is reusing the same public IP

Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Current status: PM: Deployment behind a Load-Balancer

- Additional paths: how a stateless load-balancer can pick the same server?

  ⇒ Servers: tell client not to use initial address and announce a new one.



Smartphone and WiFi icons by Blurred203 and Antü Plasma under CC-by-sa, others from Tango project, public domain

Future: Path Manager & Packet Scheduler extension

- BPF extension (`struct_ops`):
  - To adapt to specific use-cases, at a lower cost
  - Including quite a bit of clean-up in the current code!



- Support more corner cases, e.g. paths from “too heterogeneous” environments

- Better support Linux phones use-cases.

Future: in-kernel MPTCP usage

- To be used by other subsystems from within the kernel:
  - BPF sockmap
  - kTLS
  - nVME over MPTCP
  - etc.



⇒ more and more features “outside MPTCP” (harder to review)

Questions

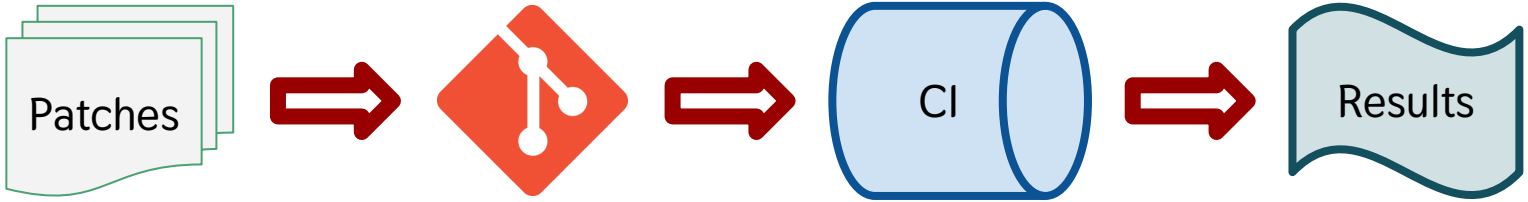
Discussions



- 🌐 mptcp.dev
- ✉️ mptcp@lists.linux.dev
- 💬 IRC: #mptcp on Libera.chat
- 🎙️ Online Meetings
- 📰 blog.mptcp.dev
- @mptcp@social.kernel.org – @matttbe@fosstodon.org

Backup

Workflow



- ML / Patchwork
- Patches in Git
- Build + tests
- Logs / Artefacts
- Failures / Instabilities
- Publications